University Scholarship Allocation

Complete worked solution

Model policy: eligibility requires aggregate >= 75%, every subject >= 60%, attendance >= 80%, and annual family income <= 600000. Award ranks are based on aggregate (descending), attendance (descending), then income (ascending), then applicant ID (ascending). The sample selects the top three eligible applicants. All values are illustrative and should be replaced by the institution's supplied rules.

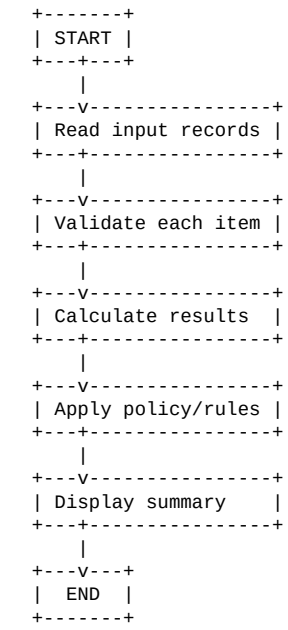
1. Computational Thinking and Algorithm

Inputs: Supplied records and policy values. **Processing:** Validate data, calculate charges or scores, apply ordered rules, and compare outcomes. **Outputs:** Itemized results, classifications, rankings, or recommendations.

Pseudocode

```
START
  Read applicant records and award limit
  FOR each applicant
    Validate marks, attendance, income
    Compute aggregate percentage
    IF every minimum and policy condition is met
      Add applicant to eligible list
  Sort eligible by aggregate descending, attendance descending,
       income ascending, applicant ID ascending
  Display ranked eligible list and selected awardees
END
```

2. Flowchart



3. Python Program

The program uses basic variables, numeric and string data, tuples, lists, loops, conditionals, arithmetic, and formatted output. Policy values are constants near the top of the program.

```
# Each record: (id, name, subject_marks, attendance_percent, annual_income)
applicants = [
    ("A104", "Maya", [88, 91, 84], 94, 420000),
    ("A102", "Noah", [79, 83, 78], 88, 510000),
    ("A103", "Ira",  [85, 86, 81], 90, 390000),
```

```
    ("A101", "Leo",  [90, 58, 92], 98, 300000),
]
minimum_subject = 60
minimum_aggregate = 75
minimum_attendance = 80
maximum_income = 600000
award_limit = 3

eligible = []
for applicant_id, name, marks, attendance, income in applicants:
    valid = len(marks) > 0 and 0 <= attendance <= 100 and income >= 0
    for mark in marks:
        if mark < 0 or mark > 100:
            valid = False
    if not valid:
        print("Invalid record:", applicant_id)
        continue
    aggregate = sum(marks) / len(marks)
    meets_subjects = True
    for mark in marks:
        if mark < minimum_subject:
            meets_subjects = False
    if (aggregate >= minimum_aggregate and meets_subjects
            and attendance >= minimum_attendance and income <= maximum_income):
        eligible.append((aggregate, attendance, income, applicant_id, name))

# Bubble sort makes every tie-break rule visible using basic loops and swaps.
for pass_number in range(len(eligible)):
    for index in range(0, len(eligible) - pass_number - 1):
        first = eligible[index]
        second = eligible[index + 1]
        swap = False
        if first[0] < second[0]:
            swap = True
        elif first[0] == second[0] and first[1] < second[1]:
            swap = True
        elif first[0] == second[0] and first[1] == second[1] and first[2] > second[2]:
            swap = True
        elif (first[0] == second[0] and first[1] == second[1]
              and first[2] == second[2] and first[3] > second[3]):
            swap = True
        if swap:
            eligible[index] = second
            eligible[index + 1] = first
print("Rank | Applicant | Aggregate | Attendance | Income")
rank = 1
for aggregate, attendance, income, applicant_id, name in eligible:
    print(rank, name, round(aggregate, 2), attendance, income)
    rank = rank + 1
print("Selected awardees:")
for row in eligible[:award_limit]:
    print(row[4], "(", row[3], ")")
```

4. Sample run guidance

Run the program as written to see a sample result. Replace the sample lists and constants with the values provided in your assignment. Keep the rule order and units consistent; for example, percentages should be entered as percentages and converted to fractions only in the calculation.